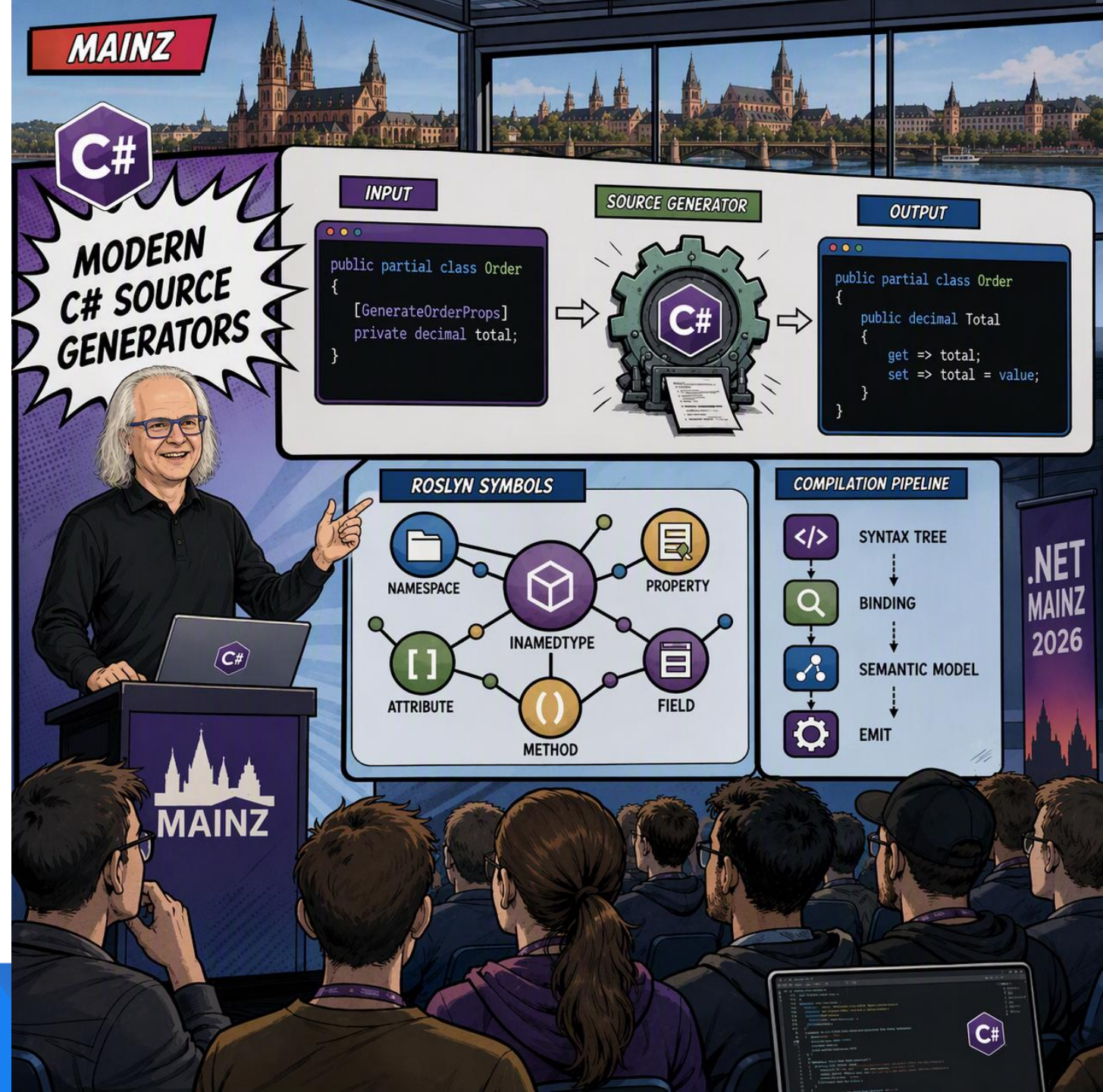


Special C# Features Used by Source Generators

Christian Nagel

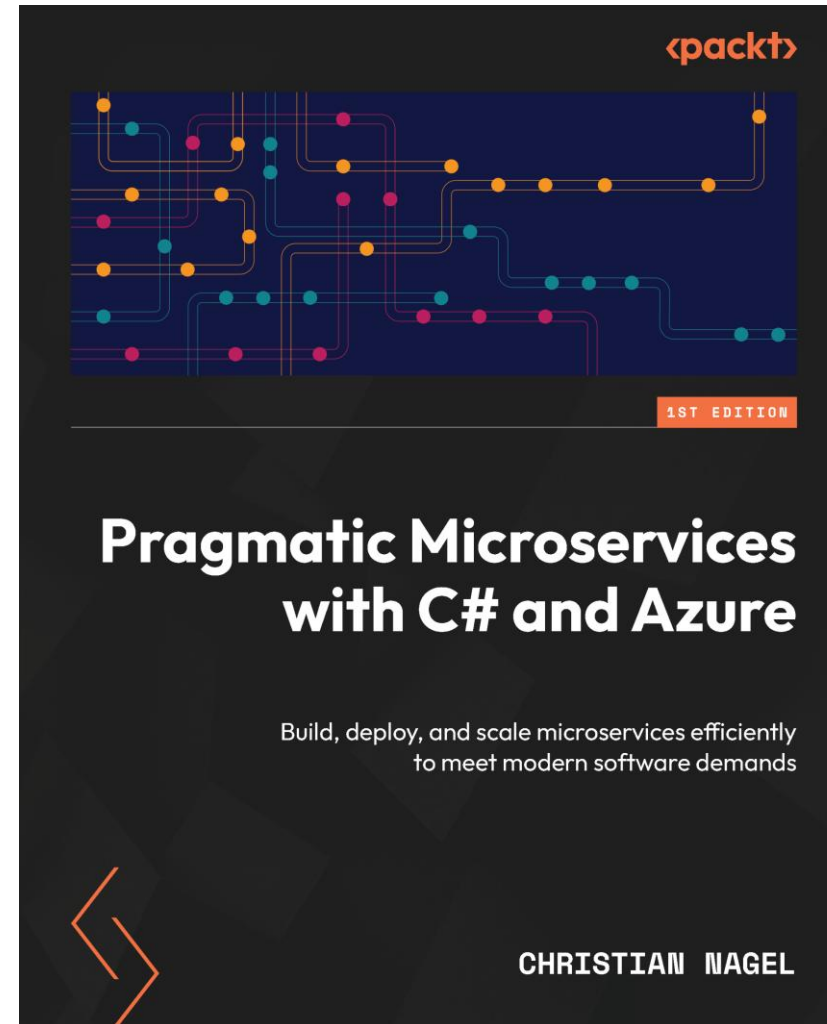
<https://www.cninnovation.com>

<https://csharp.christiannagel.com>



Christian Nagel

- Training
- Coaching
- Consulting
- Development
- New book: **Pragmatic Microservices**
- Microsoft MVP
- www.cninnovation.com
- csharp.christiannagel.com
- @christiannagel



Agenda

Source generators – why and how



Making use of

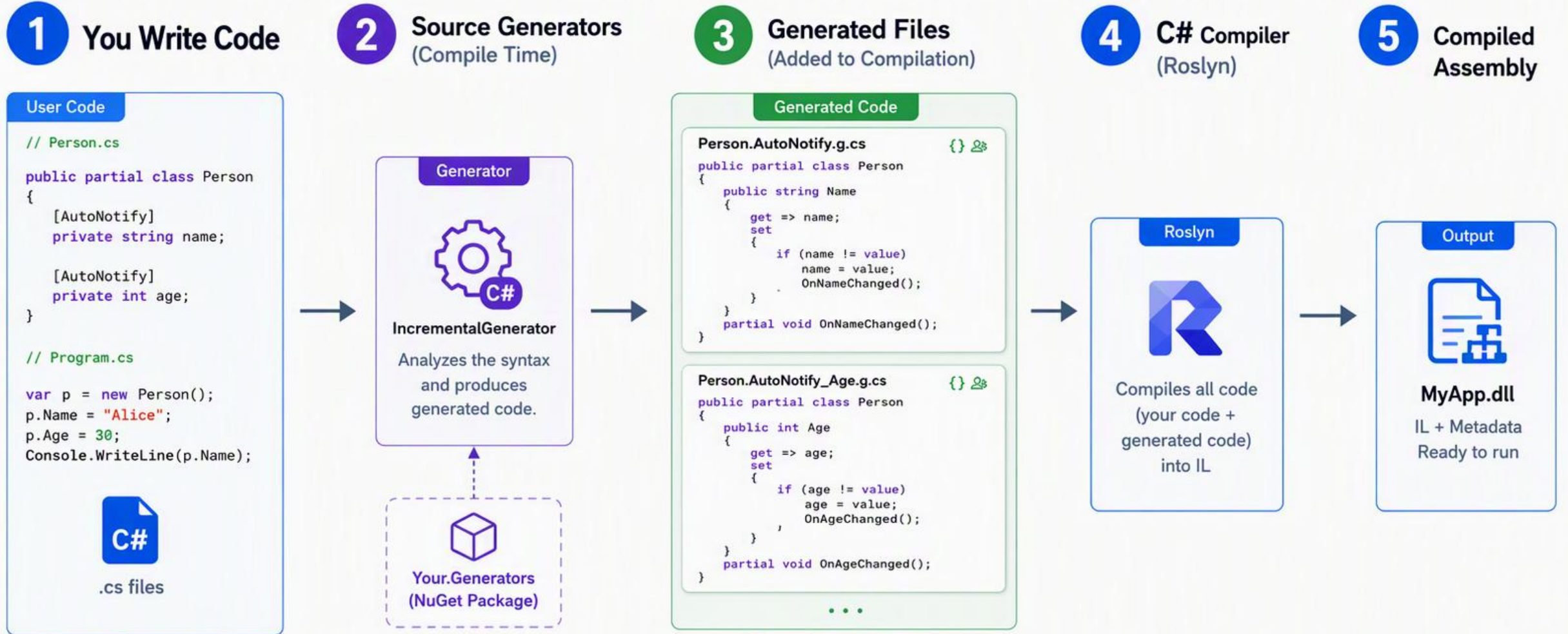
C# Partial

C# Interceptors

C# Unsafe Accessors

C# Compile-Time Code Generation

Your code + Generators → Generated code → Compiled assembly



What Happens at Compile Time


Generators inspect your code

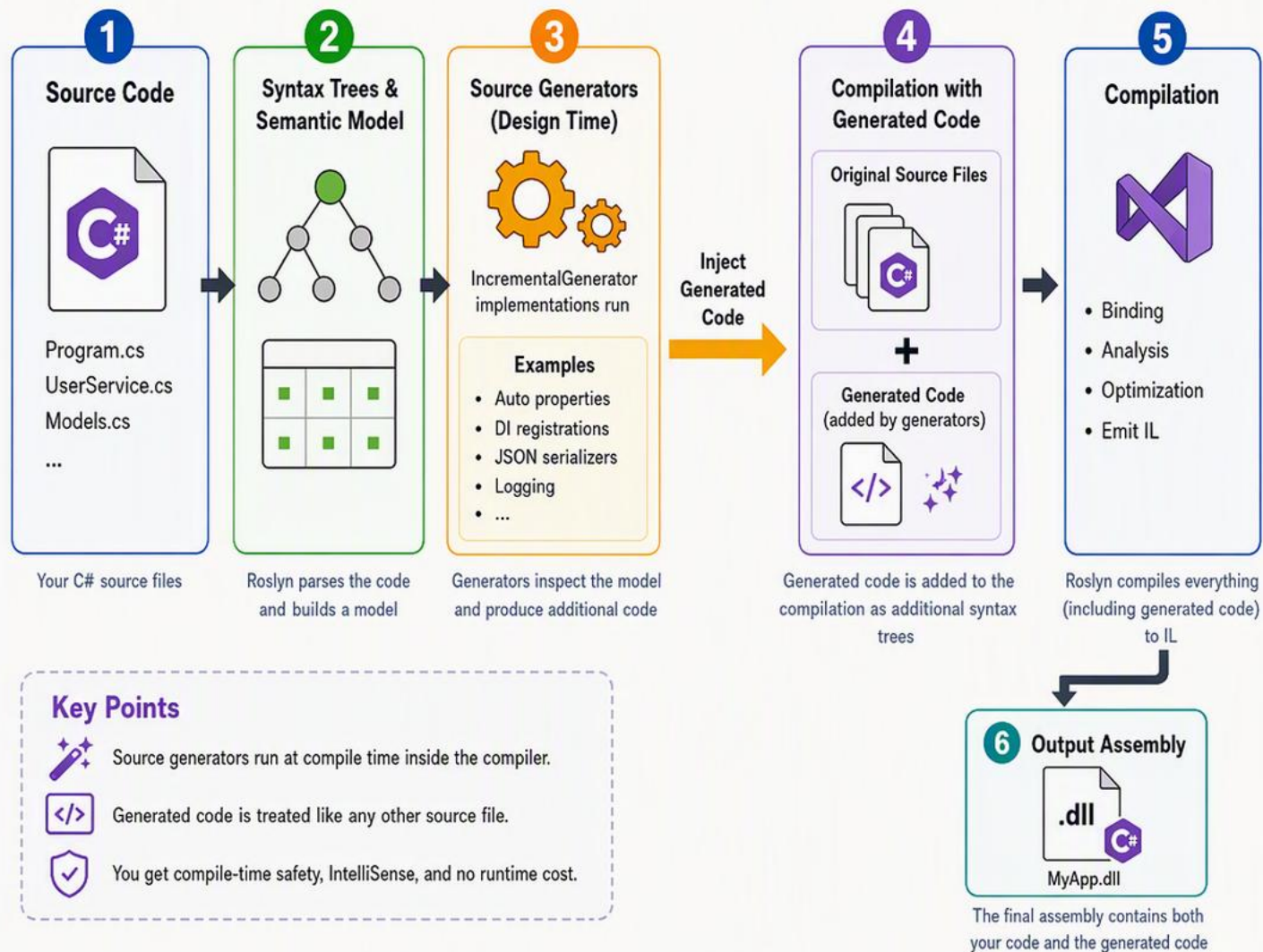

Generators produce additional source


Generated code is added to the compilation


Compiler produces the final assembly

C# Build Process with Source Generators

Source generators run during compilation and inject generated code before the compiler produces the final assembly.



In Summary

1 Source Code → 2 Syntax Trees & Model → 3 Source Generators (Inject Code) → 4 Compilation with Generated Code → 5 Compilation → 6 Output Assembly

Source generators - design goals

- Produce C# code that's added to the compilation
- Additive only – cannot change existing code
- May access additional files (also non-C# files)
- Run un-ordered – a source generator doesn't have access to output of other generators!
- User specifies generators (NuGet packages)
- Generators create a pipeline (base input → output)

Required generators

- There are exceptions (Roslyn repo)...

```
// Currently, only Razor is considered to be a required generator.  
/// </remarks>  
public static bool IsRequiredGenerator(this ISourceGenerator generator)  
{  
    // For now, we hard code the required generator list to Razor.  
    // In the future we might want to expand this to e.g. run any generators  
    // with open generated files  
    return generator.GetGeneratorType().FullName ==  
        "Microsoft.NET.Sdk.Razor.SourceGenerators.RazorSourceGenerator";  
}
```

Source generators - non-design

- Language features (e.g. records, unions)
- Code-rewriting
 - Optimization
 - Logging injection
 - IL weaving
 - ~~Call-site re-writing (Interceptors!)~~

MAKING CODE WRITE CODE. C# SOURCE GENERATOR TIME!

I'LL BUILD A
SOURCE GENERATOR
THAT CREATES
BOILERPLATE FOR ME!
LESS TYPING,
MORE SHIPPING! 🚀

PLAN

- ✓ Syntax Receiver
- ✓ Analyze
- ✓ Generate
- ✓ Profit 😊

THE GOAL:

```
[GenerateToString]
public partial class Order
{
    public int Id { get; init; }
    public string Customer { get; init; } = "";
}
```

GENERATE THIS!

```
public partial class Order
{
    public override string ToString()
    => $"Order(Id={Id}, Customer={Customer})";
}
```

STEP 1: READ THE CODE

Use a Syntax Receiver to find attributes and candidate classes.

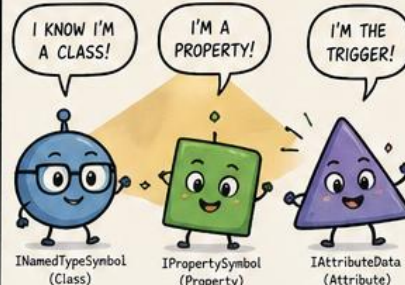
```
public class ToStringSyntaxReceiver :
    ISyntaxReceiver
{
    public List<ClassDeclarationSyntax>
        Candidates { get; } = [];

    public void OnVisitSyntaxNode(SyntaxNode node)
    {
        if (node is ClassDeclarationSyntax cds
            && cds.AttributeLists.Count > 0)
        {
            Candidates.Add(cds);
        }
    }
}
```

I SEE YOU,
[GenerateToString]!

STEP 2: GET THE MEANING (ROSLYN SYMBOLS)

SemanticModel turns syntax into rich symbols.



Symbols > Syntax
Facts over text!

STEP 3: GENERATE THE SOURCE

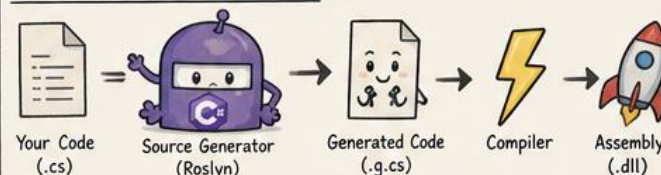
Use a SourceProductionContext to add the generated file.

```
public void Execute(GeneratorExecutionContext ctx)
{
    foreach (var model in _receiver.Candidates)
    {
        var classSymbol = (INamedTypeSymbol)
            _semantic.GetDeclaredSymbol(model)!;

        var source = GenerateToString(classSymbol);
        ctx.AddSource(
            $"{classSymbol.Name}.ToString-g.cs",
            SourceText.From(source, Encoding.UTF8));
    }
}
```

OUT COMES
A NEW .G.CS FILE!

THE MAGIC HAPPENS AT BUILD TIME ✨



BONUS
TIP!

GENERATOR CHECKLIST

- ✓ Incremental Generators for performance
- ✓ Add Analyzer (optional) to give nice diagnostics
- ✓ Test with Verify (snapshot testing FTW)

WRITE ONCE.
GENERATE FOREVER.

RESULT:

STRONGER TYPES.
LESS BOILERPLATE.
MORE TIME FOR
COOL STUFF. 😊

AUTOMATE
THE BORING
THINGS.
EMBRACE
THE GENERATOR
POWER! ⚡

BUILD SUCCEEDED ✓
0 ERRORS
0 WARNINGS
100% AWESOME

Requirements for a source generator

- Project settings
- NuGet package references
- Generator class implementing **IncrementalGenerator**
- **RegisterPostInitializationOutput** for code that runs before user code
- **Syntax tree** analysis to find relevant code
- **Semantic model** analysis to understand types and symbols

Unit tests

- Use `Microsoft.CodeAnalysis.Testing` to test generators
- Debug source generators with unit tests

Debugging a generator

- **TestHelper** invokes generator
- Uses **GeneratorDriver** to run the generator and update compilation
- Retrieves generator result

PRACTICAL SOURCE GENERATORS



Stage 1: Sample data generator

- Using the **DataSource** attribute, the source generator creates a factory of sample items

```
[DataSource(EntityName = "User", Count = 5)]
public class User
{
    public string Name { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public int Age { get; set; }
    public bool IsActive { get; set; }
    public DateTime CreatedAt { get; set; }
    public UserRole Role { get; set; }
}
```

IncrementalGeneratorInitializationContext

Compilation provider

- Access to types, symbols, metadata
- Analyze codebase

Syntax provider

- Regenerates when specific syntax changes
- Targeted for specific syntax patterns

Register post initialization output

- Output after the initialization
- Create for attributes created by the source generator

Register source output

- Connects input with output

Create attribute (marker)

- **RegisterPostInitializationOutput**
 - Create attribute that can be used by the consumer
 - Helper types, base classes, attributes
 - Emit source code once before Execute, after initialization
 - Available after the initialization before the pipeline continues

Use Syntax Provider

- **CreateSyntaxProvider**
 - Preferred way to hook into the pipeline
- **Predicate**
 - Filter and transform efficiently
 - Runs on every syntax node → fast and shallow implementation
- **Transform**
 - Deep analysis
 - Convert into meaningful data (e.g. semantic models, symbols)
 - Return information that's required for code generation
- **Where**
 - Further filter transformed results, e.g. not null
- **Collect**
 - Aggregate the data into an ImmutableArray

IncrementalValueProvider

- Declarative pipeline stage
- Computes a value once per compilation
- Lazy – only runs when needed
- Can be cached – if inputs don't change, the output is reused
- Composable – Select/Combine/Where/...

Stage 2: External files

- Add external data sources to be used by the source generator

```
[DataSource(  
    EntityName = "User",  
    Count = 5,  
    ConfigurationFile = "User.datasource.json")]
```

External files

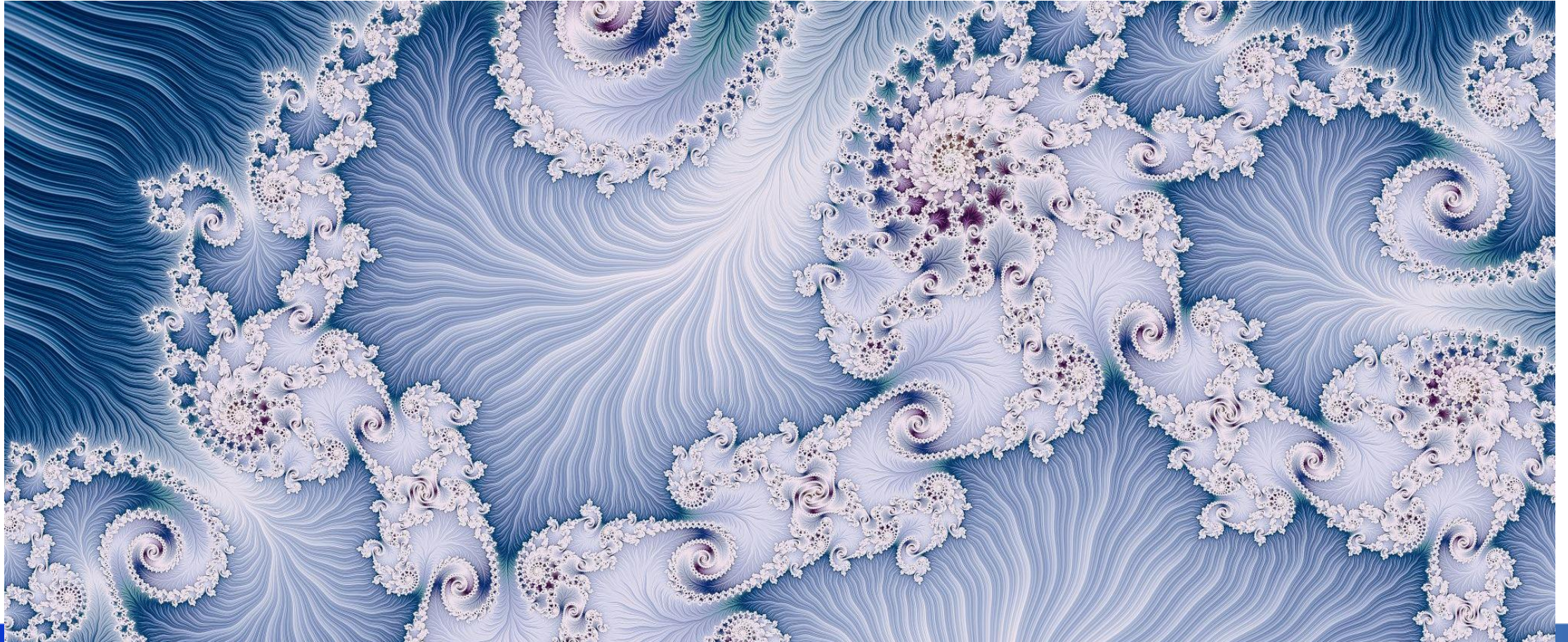
- Files need to be specified by the consumer
 - **AdditionalFiles** in project file
- **AdditionalTextProvider**
 - Access non-code files
 - You can filter file name with **Where**
- **Combine**
 - Combine two **IncrementalValueProvider** or **IncrementalValuesProvider**
 - Result is a tuple (Left, Right)
 - Use **Select** on the combination to generate output

Special update for attributes (markers)

- Typical use case
- Eliminates manual interaction with attributes
- Better caching, performance

```
var classDeclarations = context.SyntaxProvider  
    .ForAttributeWithMetadataNameC
```

More useful examples



1. Partial events

- C# 14 introduces partial events
- A generator can own the implementation behind an event declaration
- Consumers still use normal event syntax

```
public partial class EventPublisher
{
    [WeakEvent]
    public partial event Action<string> MessageReceived;
}
```

Partial events in practice

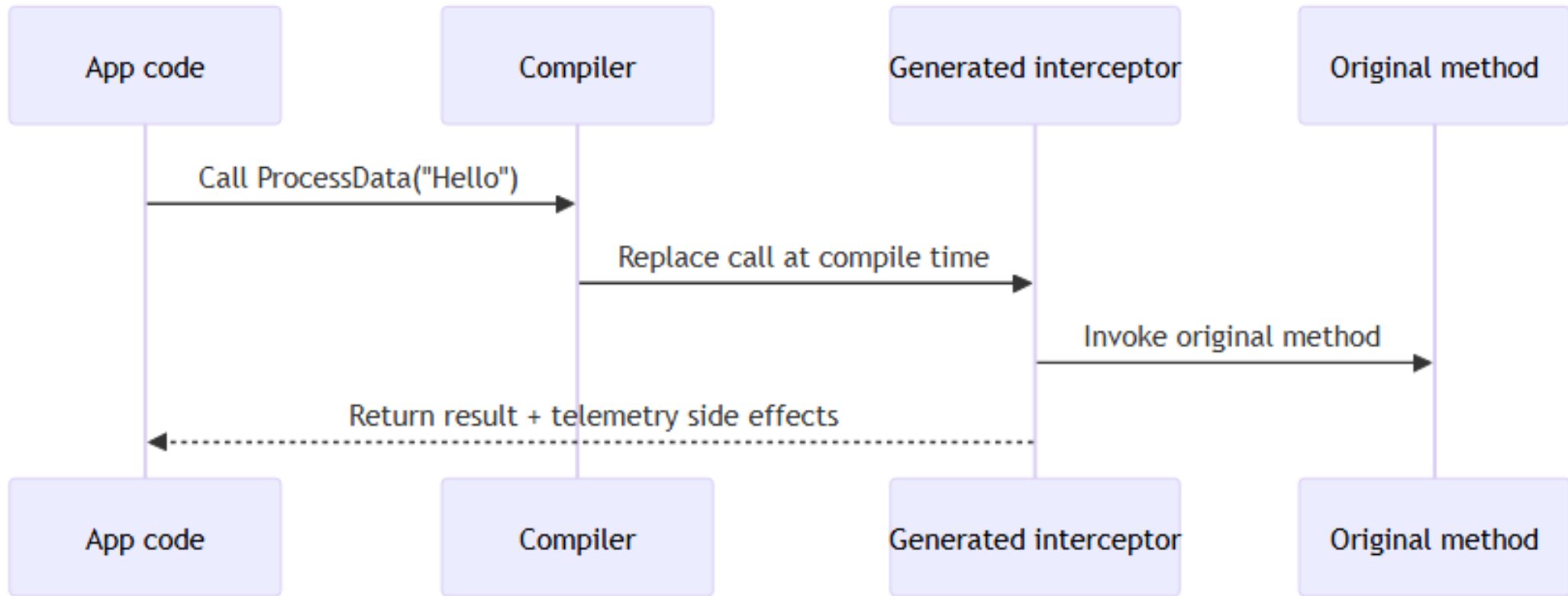
- Great for weak event patterns and other custom subscription models
- Lets generators inject add/remove logic without changing call sites
- Useful when event lifetime needs to be managed differently from standard CLR events

2. Interceptors (1)

- Interceptors let generators replace selected method calls at compile time
- Call sites stay unchanged
- Generated code can add tracing, validation, logging, or policy logic transparently

```
[Activity]  
public static string ProcessData(string input) => $"Processed: {input}";
```

2. Interceptors (2)



Interceptors in practice

- Best fit for cross-cutting concerns
- Works well when you want the generated behavior to feel invisible
- Avoids wrapper APIs like `FooWithLogging()` or `FooWithActivity()`

3. UnsafeAccessor

- UnsafeAccessor exposes private members through generated compile-time bindings
- Previously, reflection was used
- Enables high-performance access without reflection
- Useful for serializers, tests, and legacy object construction

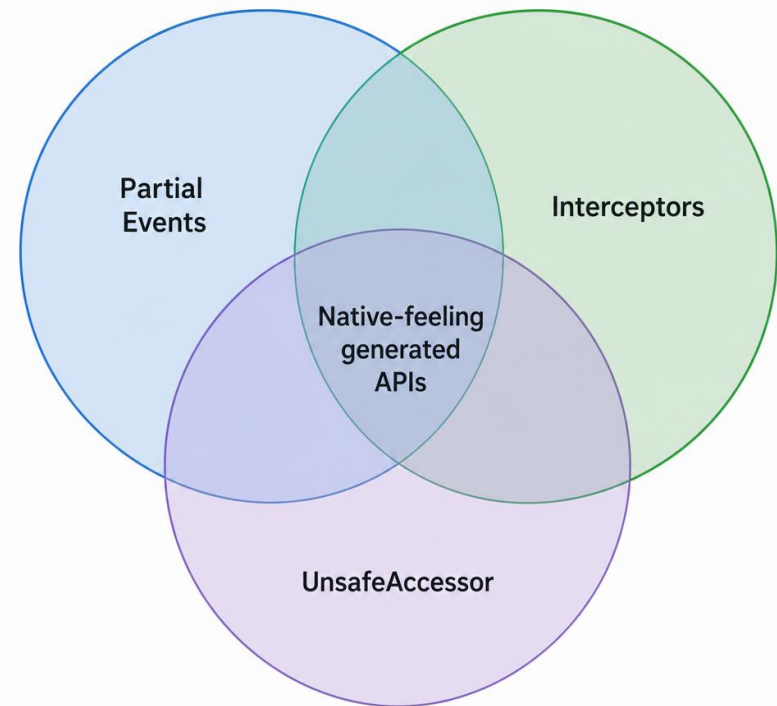
```
[UnsafeAccessor(UnsafeAccessorKind.Field, Name = "_title")]  
private static extern ref string GetTitleField(Book book);
```

UnsafeAccessor in practice

- Lets generators build fast object materializers
- Works with private fields, private setters, and private constructors
- Keeps access strongly typed while bypassing normal visibility rules

What the three features have in common

- They shift complexity from user code into generation time
- They preserve a natural API surface
- They make advanced scenarios possible without runtime frameworks or reflection-heavy code



Guidelines



Performance guidelines

- Use **CreateSyntaxProvider** with a fast predicate
 - Avoid scanning all syntax trees
- Avoid **CompilationProvider** unless needed
 - Access full compilation is expensive
 - Only use if you need semantic model-wide analysis or type resolution
- Use **Collect** only when needed
 - Overusing can increase memory pressure
- Minimize use of **AdditionalTextProvider**
 - Filter them aggressively
- Use **RegisterPostInitializationOutput** for static code
 - Emit helper types once, avoid recomputing in Execute
- Use **Combine** instead of manual combining
 - Ensures caching and invalidation

Key takeaways

- How to create source generators
- C# features used by source generators
- Partial events let generators own event behavior
- Interceptors let generators transform call sites transparently
- UnsafeAccessor lets generators reach private state safely and efficiently

Questions?

Your turn — ask questions about source generators!

Christian Nagel

<https://www.cninnovation.com>

<https://github.com/cnilearn/bastamainz2026>



We ask for your feedback

Please vote now!

